

Project 3: Hadoop and Spark

Cloud Computing (Spring 2026)

Student Name: Adewole Adeoshun

CUNY ID: 24081306

Student Type: Undergraduate Student

Professor: Jun Li

Tasks Completed: 1-7 (including Graduate Bonus)

Due Date: April 23, 2026

Cloud Provider: AWS (us-east-1)

Hadoop Version: 3.3.6

Spark Version: 3.5.5

Instance Type: t3.small (Amazon Linux 2)

TASK 1: CLUSTER SETUP

Cluster Architecture:

A 3-node cluster was provisioned on AWS EC2 in the us-east-1 (N. Virginia) region using t3.small instances running Amazon Linux 2. The cluster consists of one master node and two worker nodes, all within the same subnet for low-latency communication.

Role	Instance ID	Private IP	Public IP
Master	i-093f10e82952aaf5d	172.31.34.14	54.89.120.71
Worker 1	i-0609e411d5cc6d1d0	172.31.33.214	98.86.222.140
Worker 2	i-0fa221346b95f4467	172.31.36.71	107.21.199.217

Commands Run:

Launch 3 EC2 instances:

```
aws ec2 run-instances \  
  --region us-east-1 \  
  --image-id ami-0c02fb55956c7d316 \  
  --instance-type t3.small \  
  --count 3 \  
  --key-name redis-cluster-key \  
  --security-group-ids sg-046e972b4889f87d8
```

Security group rules (hadoop-spark-sg):

```
Port 22 - SSH access
Port 9870 - HDFS NameNode Web UI
Port 8080 - Spark Master Web UI
All traffic allowed between cluster nodes (self-referencing rule)
```

Explanation:

I used t3.small instances (2 vCPU, 2GB RAM) which were the largest free-tier eligible instance type available. All nodes are in the same Availability Zone (us-east-1f) to minimize network latency between them. A dedicated security group was created allowing inter-node communication on all ports and external access on SSH, HDFS web UI (9870), and Spark web UI (8080).

TASK 2: DEPLOY HDFS

Installation (all nodes):

```
sudo yum install -y java-11-amazon-corretto wget
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /opt/hadoop
```

Environment Variables (~/.bashrc):

```
export JAVA_HOME=/usr/lib/jvm/java-11-amazon-corretto
export HADOOP_HOME=/opt/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

core-site.xml:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://172.31.34.14:9000</value>
  </property>
</configuration>
```

hdfs-site.xml:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/opt/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/opt/hadoop/data/datanode</value>
  </property>
</configuration>
```

yarn-site.xml:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.resourcemanager.address</name>
    <value>172.31.34.14:8032</value>
  </property>
  <property>
    <name>yarn.resourcemanager.resource-tracker.address</name>
    <value>172.31.34.14:8031</value>
  </property>
</configuration>
```

workers file:

```
172.31.33.214
172.31.36.71
```

Format NameNode and start HDFS:

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

Screenshots for Task 2:

```
Configured Capacity: 1716462400 (15.98 GB)
Present Capacity: 6851557376 (5.44 GB)
DFS Remaining: 6851549184 (5.44 GB)
DFS Used: 8192 (8 KB)
DFS Used%: 0.00%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0

-----
Live datanodes (2):
Name: 172.31.33.214:9866 (ip-172-31-33-214.ec2.internal)
Hostname: ip-172-31-33-214.ec2.internal
Decommission Status : Normal
Configured Capacity: 857731200 (7.99 GB)
DFS Used: 4896 (4 KB)
Non DFS Used: 497488000 (4.58 GB)
DFS Remaining: 360242896 (3.41 GB)
DFS Used%: 0.00%
DFS Remaining%: 42.67%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xciverts: 0
Last contact: Thu Apr 23 05:19:51 UTC 2026
Last Block Report: Thu Apr 23 05:19:45 UTC 2026
Num of Blocks: 0

Name: 172.31.36.71:9866 (ip-172-31-36-71.ec2.internal)
Hostname: ip-172-31-36-71.ec2.internal
Decommission Status : Normal
Configured Capacity: 857731200 (7.99 GB)
DFS Used: 4896 (4 KB)
Non DFS Used: 6166070016 (5.76 GB)
DFS Remaining: 2391257088 (2.23 GB)
DFS Used%: 0.00%
DFS Remaining%: 27.88%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xciverts: 0
Last contact: Thu Apr 23 05:19:50 UTC 2026
Last Block Report: Thu Apr 23 05:19:45 UTC 2026
Num of Blocks: 0

[ec2-user@ip-172-31-34-14 ~]$
```

- hdfs dfsadmin -report showing 2 live DataNodes with 15.98 GB total capacity

```

[ec2-user@ip-172-31-34-14 ~]$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
[ec2-user@ip-172-31-34-14 ~]$ yarn node -list
2026-04-23 05:29:29,539 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /172.31.34.14:8032
Total Nodes:2
  Node-Id          Node-State Node-Http-Address      Number-of-Running-Containers
ip-172-31-36-71.ec2.internal:42531  RUNNING ip-172-31-36-71.ec2.internal:8042      0
ip-172-31-33-214.ec2.internal:40927  RUNNING ip-172-31-33-214.ec2.internal:8042      0
[ec2-user@ip-172-31-34-14 ~]$ █

```

- yarn node -list showing both worker nodes in RUNNING state

```

[[ec2-user@ip-172-31-34-14 ~]$ jps
10864 Jps
25749 ResourceManager
17335 Master
6813 NameNode
7118 SecondaryNameNode
[ec2-user@ip-172-31-34-14 ~]$ █

```

- jps output on master node showing NameNode, SecondaryNameNode, ResourceManager, and Spark Master running

Explanation:

Hadoop 3.3.6 was installed on all three nodes with Java 11 (Amazon Corretto). The NameNode runs on the master node and manages the HDFS namespace. The two worker nodes run DataNodes which store the actual file blocks. A replication factor of 2 was configured so each block is stored on both worker nodes, providing fault tolerance. YARN was configured with explicit ResourceManager addresses to ensure worker NodeManagers could register correctly.

TASK 3: DATA PREPARATION

Commands Run:

Download Pride and Prejudice from Project Gutenberg:

```
wget -O pride_and_prejudice.txt https://www.gutenberg.org/files/1342/1342-0.txt
```

Upload to HDFS:

```
hdfs dfs -mkdir -p /input
hdfs dfs -put pride_and_prejudice.txt /input/
hdfs dfs -ls /input/
```

Output:

```
Found 1 items
-rw-r--r--    2 ec2-user supergroup      738046 2026-04-23 05:32
/input/pride_and_prejudice.txt
```

Screenshot for Task 3:

```
[ec2-user@ip-172-31-34-14 ~]$ hdfs dfs -mkdir -p /input
[ec2-user@ip-172-31-34-14 ~]$ hdfs dfs -put pride_and_prejudice.txt /input/
[ec2-user@ip-172-31-34-14 ~]$ hdfs dfs -ls /input/
Found 1 items
-rw-r--r--    2 ec2-user supergroup      738046 2026-04-23 05:32 /input/pride_and_prejudice.txt
[ec2-user@ip-172-31-34-14 ~]$
```

- hdfs dfs -ls /input showing pride_and_prejudice.txt (738,046 bytes) stored on HDFS with replication factor 2

Explanation:

The text version of Pride and Prejudice was downloaded directly from Project Gutenberg (Book ID 1342). The file is 738 KB. It was placed in the /input directory on HDFS. The replication factor of 2 shown in the listing confirms the file is stored on both DataNodes, providing redundancy. This file serves as the input dataset for the Word Count Spark application.

TASK 4: DEPLOY APACHE SPARK

Installation (master node):

```
wget https://archive.apache.org/dist/spark/spark-3.5.5/spark-3.5.5-bin-hadoop3.tgz
tar -xzf spark-3.5.5-bin-hadoop3.tgz
sudo mv spark-3.5.5-bin-hadoop3 /opt/spark
```

Environment Variables:

```
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PYSARK_PYTHON=python3
```

spark-env.sh configuration:

```
export JAVA_HOME=/usr/lib/jvm/java-11-amazon-corretto
export HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop
export SPARK_MASTER_HOST=172.31.34.14
export SPARK_WORKER_MEMORY=1g
export SPARK_WORKER_CORES=1
```

Spark workers file:

```
172.31.33.214
172.31.36.71
```

Start Spark cluster:

```
/opt/spark/sbin/start-all.sh
```

Screenshot for Task 4:

```
[ec2-user@ip-172-31-34-14 ~]$ /opt/spark/bin/spark-submit --version
Welcome to

  ____
 /  __ \
/_ /  _/
 \  __/
  \___/   version 3.5.5

Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 11.0.30
Branch HEAD
Compiled by user ubuntu on 2025-02-23T20:30:46Z
Revision 7c29c664cdc9321205a98a14858aaf8daaa19db2
Url https://github.com/apache/spark
Type --help for more information.
[ec2-user@ip-172-31-34-14 ~]$
```

- spark-submit --version confirming Spark 3.5.5 installed with Scala 2.12.18 and Java 11

Explanation:

Spark 3.5.5 was installed on the master node and copied to both workers. The HADOOP_CONF_DIR environment variable points Spark to the Hadoop configuration files, enabling it to read and write directly to HDFS. Spark runs in Standalone mode with the master at spark://172.31.34.14:7077. Each worker was allocated 1 core and 1GB memory given the t3.small instance constraints.

TASK 5: WORD COUNT WITH SPARK

Source Code (wordcount.py):

```
from pyspark import SparkContext, SparkConf

conf = SparkConf().setAppName("WordCount")
sc = SparkContext(conf=conf)

text = sc.textFile("hdfs://172.31.34.14:9000/input/pride_and_prejudice.txt")

counts = (text
    .flatMap(lambda line: line.split())
    .map(lambda word: (word.lower().strip('.,!?";:-()[]\''), 1))
    .reduceByKey(lambda a, b: a + b)
    .filter(lambda x: x[0] != '')
    .sortBy(lambda x: x[1], ascending=False))

top20 = counts.take(20)

print("\n=== TOP 20 MOST FREQUENT WORDS ===")
for i, (word, count) in enumerate(top20, 1):
    print(f"{i:2}. {word:<20} {count}")

sc.stop()
```

Command:

```
/opt/spark/bin/spark-submit \  
  --master spark://172.31.34.14:7077 \  
  ~/wordcount.py 2>/dev/null
```

Output:

```
=== TOP 20 MOST FREQUENT WORDS ===  
1. the 4615  
2. to 4265  
3. of 3821  
4. and 3617  
5. her 2230  
6. a 2004  
7. in 1955  
8. was 1862  
9. i 1775  
10. she 1703  
11. that 1522  
12. not 1469  
13. it 1462  
14. he 1313  
15. his 1277  
16. be 1253  
17. as 1213  
18. you 1192  
19. had 1177  
20. with 1088
```

Screenshot for Task 5:

```
[ec2-user@ip-172-31-34-14 ~]$ /opt/spark/bin/spark-submit \  
> --master spark://172.31.34.14:7077 \  
[> ~/wordcount.py 2>/dev/null  
  
=== TOP 20 MOST FREQUENT WORDS ===  
1. the 4615  
2. to 4265  
3. of 3821  
4. and 3617  
5. her 2230  
6. a 2004  
7. in 1955  
8. was 1862  
9. i 1775  
10. she 1703  
11. that 1522  
12. not 1469  
13. it 1462  
14. he 1313  
15. his 1277  
16. be 1253  
17. as 1213  
18. you 1192  
19. had 1177  
20. with 1088  
[ec2-user@ip-172-31-34-14 ~]$
```

- Word Count output showing top 20 most frequent words from Pride and Prejudice

Explanation:

The Word Count program reads the file from HDFS using `sc.textFile()`. Each line is split into words using `flatMap`, then each word is lowercased and stripped of punctuation using `strip()`. The `reduceByKey` transformation sums the counts for each unique word across all partitions. The result is sorted in descending order by frequency and the top 20 are printed using `take(20)`.

The results are consistent with an English novel — common stop words like 'the', 'to', 'of', and 'and' dominate. The word 'her' ranking 5th with 2,230 occurrences is characteristic of *Pride and Prejudice*, a novel centered on female characters.

TASK 6: ESTIMATING π WITH MONTE CARLO METHOD

Source Code (montecarlo_pi.py):

```
from pyspark import SparkContext, SparkConf
import random

conf = SparkConf().setAppName("MonteCarloPi")
sc = SparkContext(conf=conf)

NUM_SAMPLES = 10000000

def inside(p):
    x, y = random.random(), random.random()
    return x*x + y*y < 1.0

count = sc.parallelize(range(NUM_SAMPLES), 4) \
    .filter(inside) \
    .count()

pi_estimate = 4.0 * count / NUM_SAMPLES

print(f"\n=== MONTE CARLO PI ESTIMATION ===")
print(f"Number of samples:      {NUM_SAMPLES:,}")
print(f"Points inside circle: {count:,}")
print(f"Estimated Pi:             {pi_estimate:.6f}")
print(f"Actual Pi:                 3.141593")
print(f"Error:                     {abs(pi_estimate - 3.141593):.6f}")

sc.stop()
```

Command:

```
/opt/spark/bin/spark-submit \
--master spark://172.31.34.14:7077 \
~/montecarlo_pi.py 2>/dev/null
```

Output:

```
=== MONTE CARLO PI ESTIMATION ===
Number of samples:      10,000,000
Points inside circle: 7,854,992
Estimated Pi:          3.141997
Actual Pi:              3.141593
Error:                  0.000404
```

Screenshot for Task 6:

```
[ec2-user@ip-172-31-34-14 ~]$ /opt/spark/bin/spark-submit \
> --master spark://172.31.34.14:7077 \
[> ~/montecarlo_pi.py 2>/dev/null
```

```
=== MONTE CARLO PI ESTIMATION ===
Number of samples: 10,000,000
Points inside circle: 7,854,992
Estimated Pi: 3.141997
Actual Pi: 3.141593
Error: 0.000404
[ec2-user@ip-172-31-34-14 ~]$
```

- Monte Carlo Pi estimation result — 10 million samples, estimated Pi = 3.141997 (error: 0.000404)

Explanation:

The Monte Carlo method estimates Pi by exploiting the relationship between a unit circle (area = π) and the 2x2 square containing it (area = 4). Random points (x, y) are uniformly generated in the unit square [0,1] x [0,1]. A point falls inside the quarter circle if $x^2 + y^2 < 1$. The proportion of points inside multiplied by 4 gives an estimate of Pi.

Spark parallelizes this by distributing the 10 million samples across 4 partitions using `sc.parallelize()`. Each partition independently generates and tests its share of random points using the `filter()` transformation. The `count()` action aggregates the total points inside across all partitions. With 10 million samples the estimate of 3.141997 is accurate to within 0.013% of the true value.

TASK 7: PERFORMANCE BENCHMARKING & SCALABILITY ANALYSIS

(Undergraduate Bonus +10 points)

Benchmark Script (benchmark.py):

```
from pyspark import SparkContext, SparkConf
import time, sys

config_name = sys.argv[1]
conf = SparkConf().setAppName(f"WordCount_Benchmark_{config_name}")
sc = SparkContext(conf=conf)

start = time.time()
text = sc.textFile("hdfs://172.31.34.14:9000/input/pride_and_prejudice.txt")
counts = (text
    .flatMap(lambda line: line.split())
    .map(lambda word: (word.lower().strip('.,!?"';:-()[]\''), 1))
    .reduceByKey(lambda a, b: a + b)
    .filter(lambda x: x[0] != '')
    .sortBy(lambda x: x[1], ascending=False))
top20 = counts.take(20)
```

```
elapsed = time.time() - start

print(f"\n=== BENCHMARK: {config_name} ===")
print(f"Execution time: {elapsed:.2f} seconds")
sc.stop()
```

Configurations Tested:

Config	Executors	Memory	Cores	Wall Time	App Time
Config 1	1	512 MB	1	26.18s	16.67s
Config 2	2	512 MB	1	25.07s	16.42s
Config 3	2	1 GB	1	16.04s	8.54s

Commands:

```
time /opt/spark/bin/spark-submit \
  --master spark://172.31.34.14:7077 \
  --num-executors 1 --executor-memory 512m --executor-cores 1 \
  ~/benchmark.py "1executor_512m" 2>/dev/null

time /opt/spark/bin/spark-submit \
  --master spark://172.31.34.14:7077 \
  --num-executors 2 --executor-memory 512m --executor-cores 1 \
  ~/benchmark.py "2executors_512m" 2>/dev/null

time /opt/spark/bin/spark-submit \
  --master spark://172.31.34.14:7077 \
  --num-executors 2 --executor-memory 1g --executor-cores 1 \
  ~/benchmark.py "2executors_1g" 2>/dev/null
```

Screenshot for Task 7:

```
[ec2-user@ip-172-31-34-14 ~]$ time /opt/spark/bin/spark-submit \
> --master spark://172.31.34.14:7077 \
> --num-executors 1 \
> --executor-memory 512m \
> --executor-cores 1 \
|> ~/benchmark.py "1executor_512m" 2>/dev/null

=== BENCHMARK: 1executor_512m ===
Execution time: 16.67 seconds
Top word: ('the', 4615)

real    0m26.179s
user    0m20.773s
sys      0m0.691s
[ec2-user@ip-172-31-34-14 ~]$ time /opt/spark/bin/spark-submit \
> --master spark://172.31.34.14:7077 \
> --num-executors 2 \
> --executor-memory 512m \
> --executor-cores 1 \
|> ~/benchmark.py "2executors_512m" 2>/dev/null

=== BENCHMARK: 2executors_512m ===
Execution time: 16.42 seconds
Top word: ('the', 4615)

real    0m25.074s
user    0m20.524s
sys      0m0.660s
[ec2-user@ip-172-31-34-14 ~]$ time /opt/spark/bin/spark-submit \
> --master spark://172.31.34.14:7077 \
> --num-executors 2 \
> --executor-memory 1g \
> --executor-cores 1 \
|> ~/benchmark.py "2executors_1g" 2>/dev/null

=== BENCHMARK: 2executors_1g ===
Execution time: 8.54 seconds
Top word: ('the', 4615)

real    0m16.041s
user    0m17.281s
sys      0m0.728s
[ec2-user@ip-172-31-34-14 ~]$
```

- Benchmark results for all 3 configurations showing execution times

Analysis:

Config 1 vs Config 2: Doubling the number of executors from 1 to 2 (both at 512MB) produced almost no speedup — wall time dropped only from 26.18s to 25.07s (4% improvement). This indicates that at 512MB memory, executors are memory-constrained and spend significant time on garbage collection and spilling data to disk, negating the benefit of parallelism. This is consistent with Amdahl's Law — when the bottleneck is a resource constraint rather than CPU parallelism, adding more workers yields diminishing returns.

Config 2 vs Config 3: Keeping 2 executors but doubling memory from 512MB to 1GB produced a dramatic improvement — wall time dropped from 25.07s to 16.04s (36% improvement) and application time dropped from 16.42s to 8.54s (48% improvement). This confirms that memory was the primary bottleneck, not parallelism. With sufficient memory, Spark can cache shuffle data in-memory instead of spilling to disk, dramatically reducing I/O overhead.

These results align with Gustafson's Law in the sense that scaling resources (memory) scaled performance, but do not show linear speedup with executor count. The overhead of job scheduling, data serialization, and network shuffle introduces a serial fraction that limits speedup regardless of parallelism — consistent with Amdahl's Law. For a dataset the size of Pride and Prejudice (738 KB), the overhead of distributed coordination dominates and a single well-resourced executor is nearly as fast as two under-resourced ones.

Bottlenecks identified: (1) Memory pressure at 512MB causing GC overhead. (2) Small dataset size means Spark's parallelism overhead is proportionally large. (3) t3.small instances have limited network bandwidth which constrains shuffle performance.

ADDITIONAL EVIDENCE

```
[ec2-user@ip-172-31-34-14 ~]$ hdfs dfs -ls /input/
Found 1 items
-rw-r--r--    2 ec2-user supergroup      738046 2026-04-23 05:32 /input/pride_and_prejudice.txt
[ec2-user@ip-172-31-34-14 ~]$ hdfs dfsadmin -report | head -20
Configured Capacity: 17154662400 (15.98 GB)
Present Capacity: 5210529792 (4.85 GB)
DFS Remaining: 5209010176 (4.85 GB)
DFS Used: 1519616 (1.45 MB)
DFS Used%: 0.03%
Replicated Blocks:
    Under replicated blocks: 0
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
-----
[ec2-user@ip-172-31-34-14 ~]$
```

- HDFS listing showing pride_and_prejudice.txt and dfsadmin report confirming 15.98 GB cluster storage

```
[ec2-user@ip-172-31-34-14 ~]$ yarn node -list
2026-04-23 16:28:43,864 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at /172.31.34.14:8032
Total Nodes:2
  Node-Id          Node-State Node-Http-Address      Number-of-Running-Containers
ip-172-31-36-71.ec2.internal:42531  RUNNING ip-172-31-36-71.ec2.internal:8042  0
ip-172-31-33-214.ec2.internal:40927  RUNNING ip-172-31-33-214.ec2.internal:8042  0
[ec2-user@ip-172-31-34-14 ~]$
```

- Final yarn node -list confirmation , both worker nodes RUNNING at end of session

SUMMARY

This project successfully deployed a full Hadoop and Spark data analytics stack on AWS EC2 and executed all required applications:

- Cluster Setup: 3-node AWS cluster (1 master + 2 workers) using t3.small instances in us-east-1.
- HDFS Deployment: Hadoop 3.3.6 installed across all nodes with NameNode, 2 DataNodes, and YARN ResourceManager/NodeManagers all running correctly.
- Data Preparation: Pride and Prejudice (738 KB) downloaded from Project Gutenberg and stored on HDFS with replication factor 2.
- Spark Deployment: Spark 3.5.5 configured in Standalone mode with HDFS integration via HADOOP_CONF_DIR.

- Word Count: PySpark application correctly counted, sorted, and output the top 20 most frequent words. Top word: 'the' (4,615 occurrences).
- Monte Carlo Pi: 10 million random samples distributed across 4 Spark partitions estimated Pi as 3.141997 (error: 0.000404).
- Performance Benchmarking (Bonus): 3 configurations tested. Memory was identified as the primary bottleneck — doubling memory from 512MB to 1GB improved execution time by 48%, while doubling executors at 512MB improved it by only 4%. Results discussed in context of Amdahl's Law.